Mini Project 2 · UART · FIFO · Loopback

UART_FIFO Loopback

2조 정준혁

2026.10.26.

Contents

01 개요

02 설계 기준

03 모듈 구조

04 RTL Schematic

05 타이밍 다이어그램

06 Simulation 시나리오

07 Simulation 결과 — FIFO · Loopback · 랜덤

08 시연 영상

09 결론 및 고찰

1. 개요

결론 **UART 루프백에 FIFO 2단 버퍼 적용 — 랜덤 16 byte PASS 16 / FAIL 0, 정상 동작**

Tool Vivado 2020.2 · XSim **Board** Digilent Basys 3 **FPGA** Artix-7 xc7a35tcpg236-1

설계 대상

모듈	역할
baud_tick_gen	9600 bps × 16 tick 생성
uart_rx	직렬 → 병렬 변환
fifo × 2	4-depth 수신 / 송신 버퍼
uart_tx	병렬 → 직렬 변환

Top 입출력

포트	방향	설명
clk	in	100 MHz 클럭
rst	in	리셋 (active high)
rx	in	PC → FPGA 직렬
tx	out	FPGA → PC 직렬

Note. 통신 형식 9600 bps · 8N1 (Start 1 · Data 8 · Stop 1)

2. 설계 기준

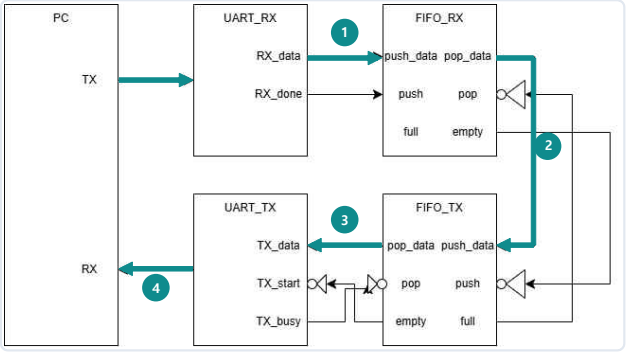
결론 **100 MHz 보드 클럭으로 9600 bps를 오차 0.006%로 생성 — TB 비트 주기도 DUT와 일치**

항목	설정값	설정 이유
시스템 클럭	100 MHz	Basys 3 내장 오실레이터 (W5) — 별도 PLL 없이 사용
Baud rate	9600 bps	PC 터미널 표준 속도 · 저속이라 타이밍 여유 큼
프레임	8N1 (10 bit)	UART 표준 형식, 1 frame ≈ 1.04 ms
오버샘플링	16× (F_COUNT = 651)	비트 중앙(8 tick)에서 샘플 → 클럭 오차 · 노이즈에 강함
Baud 오차	9600.6 bps (+0.006%)	100 MHz ÷ (651 × 16) — UART 허용 오차(통상 ±2%) 대비 충분히 작음
FIFO DEPTH	4 (포인터 2 bit)	2의 거듭제곱 → 포인터 자연 순환 · 루프백은 송수신 속도가 같아 소용량으로 충분
TB 비트 주기	104,160 ns	10,416 clk × 10 ns = DUT 1 bit (651 × 16 clk)와 동일

Note. 1 bit = 16 tick = 10,416 clk · F_COUNT = 100,000,000 / (9600 × 16)

3. 모듈 구조

결론 **FIFO 2개를 사이에 둔 5단 구조 — 핸드셰이크는 empty · full · busy 반전 신호**

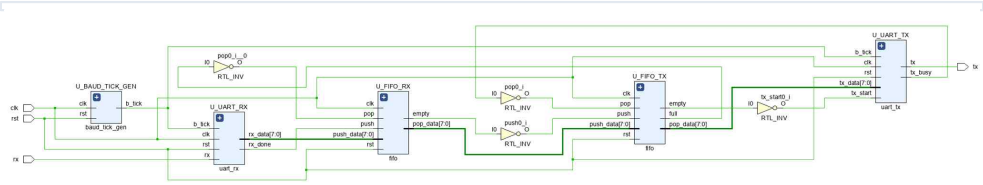


데이터 경로 (8 bit)

No	구간
1	UART_RX → FIFO_RX push = rx_done
2	FIFO_RX → FIFO_TX push = ~empty · pop = ~full
3	FIFO_TX → UART_TX tx_start = ~empty · pop = ~tx_busy
4	UART_TX → PC Start · Data 8 · Stop 직렬 송신

4. RTL Schematic

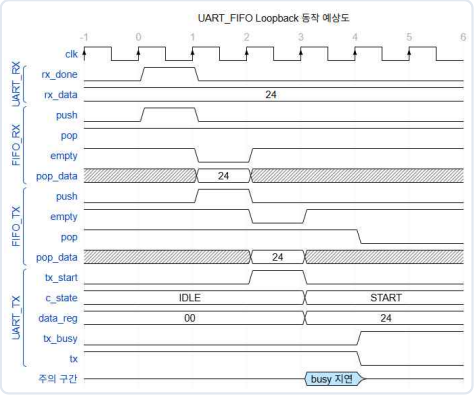
결론 합성 RTL이 설계 구조와 일치 — 모듈 5개 + 반전 게이트(RTL_INV) 4개



구분	수신부	FIFO_RX	FIFO_TX	송신부
모듈	baud_tick_gen · uart_rx	fifo (DEPTH 4)	fifo (DEPTH 4)	uart_tx
입력 조건	rx 직렬 입력	push = rx_done	push = ~empty(RX)	tx_start = ~empty(TX)
출력 조건	rx_data · rx_done	pop = ~full(TX)	pop = ~tx_busy	tx_data = pop_data

5. 타이밍 다이어그램

결론 **rx_done(E0) → UART_TX 송신 시작(E3)까지 3클럭 (30 ns) — 핸드셰이크 정상 동작**



엣지	시각	동작
E0	996,045 ns	rx_done = 1 (수신 완료) → FIFO_RX push
E1	996,055 ns	FIFO_RX ← 24, empty 0 → FIFO_TX push = 1
E2	996,065 ns	FIFO_RX pop, FIFO_TX ← 24 → tx_start = 1
E3	996,075 ns	UART_TX data_reg ← 24 IDLE → START, FIFO_TX pop
E4	996,085 ns	tx_busy = 1 → FIFO_TX pop = 0 tx Start bit 출력

Note. 상단 숫자 0 ~ 4 = E0 ~ E4 · 1 clk = 10 ns · WaveDrom 동작 예상도 (시뮬레이션 파형과 일치)

6. Simulation 시나리오 설명

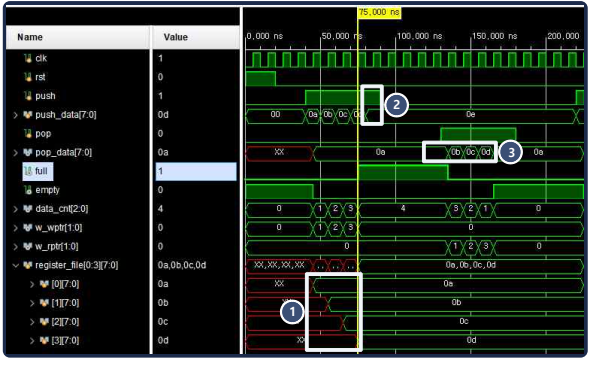
결론 **Test Case별 입력 → 확인 항목 → 예상 결과를 정의하고 파형으로 검증**

No	검증 항목	Input Stimulus	Checklist (관찰 신호)	Expectation
1	**FIFO 단독** (tb_fifo_ppt)	Reset → push 5회 (0a ~ 0e) → pop 5회 → push · pop 동시 5회 (0f ~ 0b)	push, pop, full, empty w_wptr, w_rptr, pop_data	4개 저장 후 full, 5번째 push 무시 0a → 0d 순서 출력 후 empty **full 시 push · empty 시 pop 무시**
2	**Loopback 1 byte** (tb_uart)	Reset → sender_uart(8'h24) rx에 Start · Data 8 · Stop 인가	rx_done, push, pop, empty register_file, tx_start, tx_busy, data_reg	UART_RX → FIFO_RX → FIFO_TX → UART_TX 3클럭 내 전달 **data_reg = 24, tx로 같은 프레임 송신**
3	**Loopback 랜덤** (tb_uart)	$random 16 byte sender_uart → receive_uart → score_board 반복	expected_data, result_data pass_count, fail_count	result = expected 16회 **PASS 16 / FAIL 0**
4	**보드 시연**	ComPort Master로 문자 · 16진수 전송	PC 수신 창	**보낸 값이 그대로 에코**

- **판정 :** $display 출력(pass / fail count)과 파형의 Hex 값을 Expectation과 비교
- **공통 조건 :** 100 MHz clk, 9600 bps 8N1 (BAUD_BIT_PERIOD = 104,160 ns), FIFO DEPTH = 4

7. Simulation — FIFO 동작

결론 **FIFO 정상 동작 — 4개 적재 후 full, 저장 순서대로 출력 · random 유효 pop 113회 PASS / FAIL 0**



1 Push only — 4개 적재

wptr 0 → 3, register_file[0 ~ 3]에 0a ~ 0d 저장
75 ns : wptr = rptr → full = 1

2 Full 상태의 push

full 중 들어온 0e push 무시
wptr · register_file 유지 → overflow 방지

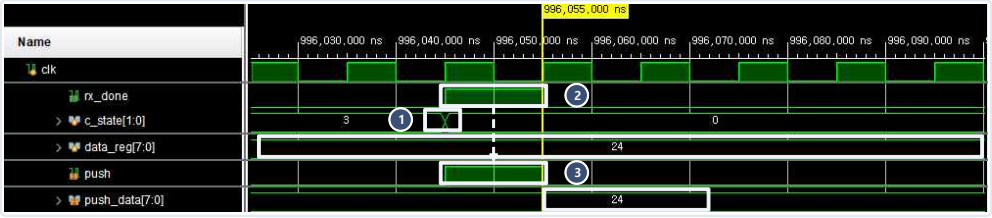
3 Pop only — 순서대로 출력

rptr 0 → 3, 저장 순서대로 0a → 0d 출력
rptr = wptr → empty = 1 (underflow 방지)

Note. 노란 커서 = 75 ns (full 진입) · data_cnt = TB에서 계산한 저장 개수 · empty 상태의 pop_data(0a)는 이전 값이 남은 무효 데이터 (조합 읽기)

7. Simulation — Loopback ①-1 UART_RX 수신

결론 **정상 동작 — 수신 완료(E0) 시 rx_done 1클럭 펄스가 FIFO_RX push로 인가, rx_data = 8'h24**

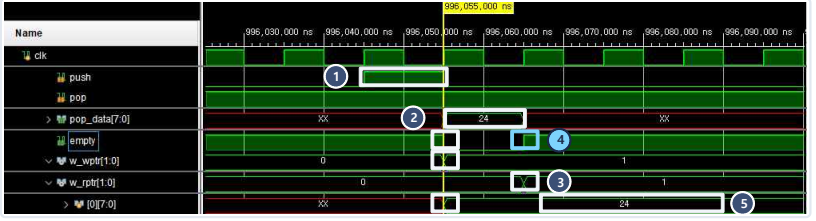


No	시점	동작
1	수신	PC가 보낸 직렬 데이터를 UART_RX가 수신 → data_reg = 8'h24로 복원
2	E0 · 완료	STOP 처리 후 c_state 3 → 0 (IDLE), rx_done 1클럭 펄스 발생
3	E0 ~ E1	rx_done → FIFO_RX push = 1, rx_data(24) → FIFO_RX push_data

Note. E0 = rx_done 상승 엣지 · 1클럭 = 10 ns · 노란 커서 = E1 (996,055 ns)

7. Simulation — Loopback ①-2 FIFO_RX 저장 → 전달

결론 정상 동작 — E1에 저장 · E2에 FIFO_TX로 전달, FIFO_RX empty = 0은 1클럭만 유지

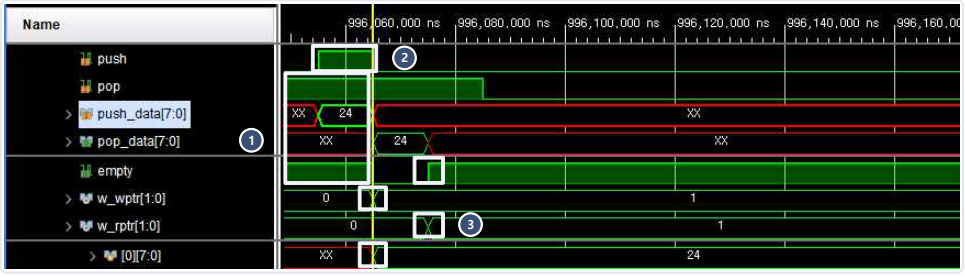


No	시점	동작
1	E1 · push	rx_data(24) → w_wptr가 가리키는 register_file[0]에 저장, w_wptr 0 → 1
2	E1 · empty	데이터 적재 → empty 1 → 0, 조합 읽기라 pop_data = 24 즉시 출력
3	E2 · pop	pop(= ~full(TX), 항상 1) 처리 → w_rptr 0 → 1, 같은 엣지에 FIFO_TX가 24 저장
4	E2 · empty	w_rptr = w_wptr → empty 다시 1 (empty = 0은 1클럭 동안만 유지)
5	E2 이후	register_file[0] = 24 유지 — rptr가 지난 값은 무효, 다음 push 때 덮어씀

Note. FIFO_TX push = ~empty(RX) · FIFO_RX pop = ~full(TX) → 평소 항상 1 · empty 상태의 pop은 control_unit이 무시

7. Simulation — Loopback ②-1 FIFO_TX 저장 → pop

결론 정상 동작 — E2에 FIFO_TX 저장, 1클럭 뒤(E3) pop · empty 중 pop은 무시

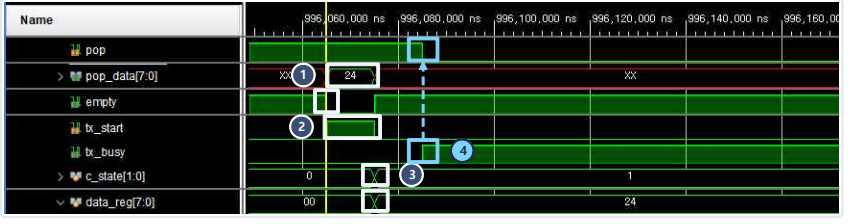


No	시점	동작
1	E2 이전	FIFO_TX empty = 1, UART_TX 유휴 → pop = 1이지만 empty라 무시 → rptr 유지, pop_data 무효
2	E2 · push	FIFO_RX empty = 0 → FIFO_TX push = 1 → push_data(24)를 register_file[0]에 저장, wptr 0 → 1
3	E3 · pop	계속 1이던 pop 처리 → 저장 1클럭 뒤 내보냄, rptr 0 → 1, empty 1 복귀

Note. E2는 push · pop 동시 입력(2'b11) · empty 상태 → push만 처리

7. Simulation — Loopback ②-2 UART_TX 송신 시작

결론 **정상 동작 — E3에 UART_TX가 24 캡처, E4부터 tx_busy로 pop 차단 (rx_done 후 3클럭)**



No	시점	동작
1	E2 ~ E3	FIFO_TX pop_data(24) → UART_TX tx_data로 직접 연결
2	E2 · tx_start	FIFO_TX empty 1 → 0 → tx_start(= ~empty) = 1, UART_TX 동작 시작
3	E3 · 캡처	IDLE에서 tx_start 감지 → data_reg ← 24, c_state IDLE → START
4	E4 · tx_busy	tx_busy 0 → 1 → pop(= ~tx_busy) = 0, 송신 중 다음 데이터 pop 차단
5	정리	tx_busy = 0(유휴)일 때만 pop 활성 → 송신 완료 후 다음 pop_data 전달

Note. tx_start = ~empty(TX) · pop = ~tx_busy · tx_busy는 레지스터 출력 → START 진입 1클럭 뒤 상승 (9. 고찰)

7. Simulation — 랜덤 카운트

결론 **랜덤 16 byte 루프백 PASS 16 / FAIL 0 — 손실 · 변형 없이 정상 동작**



☐ expected 송신 → 다음 구간에 같은 값 result 수신 ■ fail_count = 0 유지

#	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
송신	24	81	09	63	0d	8d	65	12	01	0d	76	3d	ed	8c	f9	c6
수신	24	81	09	63	0d	8d	65	12	01	0d	76	3d	ed	8c	f9	c6

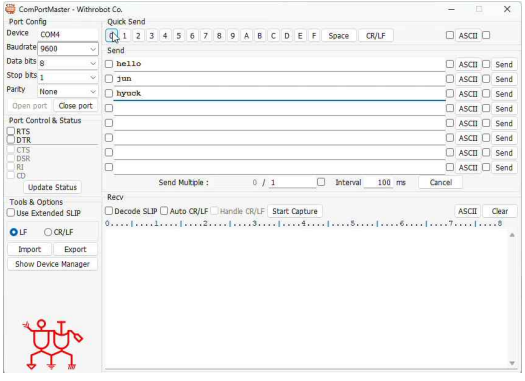
16 / 16 PASS

sender_uart	$random byte → rx에 Start · Data 8 (LSB first) · Stop, 1 bit = 104,160 ns
receive_uart	tx 하강 감지 → 반 비트 후 Start 확인 → 1 bit 간격 8회 샘플 → Stop 확인
score_board	result = expected → pass_count + 1, 다르면 fail_count + 1

Note. result_data는 수신 완료 시점에 갱신 → 파형에서 expected보다 한 구간 늦게 표시

8. 시연 영상 — ComPort Master

결론 **보드 정상 동작 — ComPort Master로 보낸 문자 · 16진수가 그대로 에코**



1 시연 환경

Basys 3 ↔ PC, 9600-8-N-1

2 확인 항목

보낸 문자가 그대로 에코

3 결과

16진수 및 문자 출력 확인

9. 결론 및 고찰

결론 **전 항목 정상 동작 — FIFO FAIL 0, 루프백 16 / 16 PASS, 보드 에코 확인**

결론

검증 항목	결과 (수치)	판정
FIFO 단독	push 4개 후 full, 0a → 0d 순서 출력 후 empty	**정상**
Loopback 1 byte	rx_done → UART_TX 송신 시작 3 clk (30 ns)	**정상**
Loopback 랜덤	16 byte PASS 16 / FAIL 0	**정상**
보드 시연	ComPort Master 문자 · 16진수 에코	**정상**

고찰

구분	현 시점 Issue	향후 계획
기능 확장	받은 데이터를 PC로 다시 보내는 에코만 함	수신한 문자를 FND에 출력하여 pc화면과 보드에서 동시에 확인 가능하게 연동
검증 범위	1 byte씩 송수신 → FIFO 최대 1 byte만 저장 → full · overflow 미검증	5 byte 이상 연속 송신 TB 추가 → full 시 push 무시 확인